Java Backend Architecture

Interview Series

Chapter 7.4
Maven Multi-Module Projects

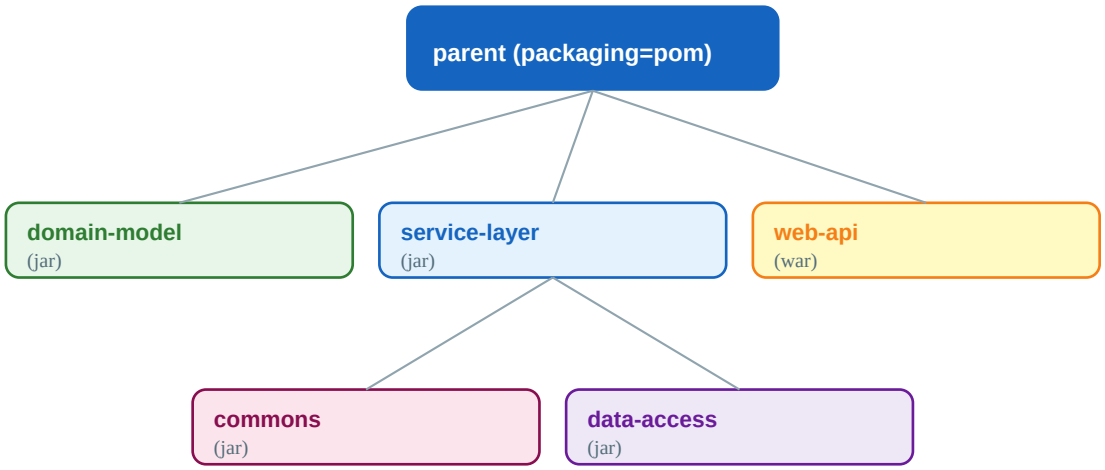
50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 7.4 — Maven Multi-Module Projects

Introduction

Multi-module (aggregator) projects allow large codebases to be split into smaller, independently versioned modules while still being built together as a unit. Maven's Reactor handles build ordering, inter-module dependency resolution, and partial builds. Understanding parent POM inheritance, aggregation, and the reactor is critical for enterprise Java architectures.

Project Structure



Reactor builds in dependency order: commons → domain-model → data-access → service-layer → web-api

Parent POM (Aggregator)

```
<!-- parent/pom.xml — packaging must be "pom" --> <project> <modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId> <artifactId>my-app-parent</artifactId>
<version>1.0.0-SNAPSHOT</version> <packaging>pom</packaging> <!-- REQUIRED for aggregator -->
<!-- List of sub-modules (relative directory paths) --> <modules> <module>commons</module>
<module>domain-model</module> <module>data-access</module> <module>service-layer</module>
<module>web-api</module> </modules> <!-- Centralised dependency versions -->
<dependencyManagement> <dependencies> <dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-dependencies</artifactId> <version>3.2.0</version> <type>pom</type>
<scope>import</scope> </dependency> </dependencies> </dependencyManagement> <!-- Centralised
plugin config --> <build> <pluginManagement> <plugins> <plugin>
<groupId>org.apache.maven.plugins</groupId> <artifactId>maven-compiler-plugin</artifactId>
<version>3.12.1</version> <configuration><release>17</release></configuration> </plugin>
</plugins> </pluginManagement> </build> </project>
```

Child Module POM

```
<!-- service-layer/pom.xml --> <project> <modelVersion>4.0.0</modelVersion> <!-- Reference to
parent --> <parent> <groupId>com.example</groupId> <artifactId>my-app-parent</artifactId>
<version>1.0.0-SNAPSHOT</version> <!-- relativePath optional — default is ../pom.xml -->
<relativePath>../pom.xml</relativePath> </parent> <!-- groupId and version inherited from
parent (can override) --> <artifactId>service-layer</artifactId> <!-- packaging defaults to jar
--> <dependencies> <!-- Inter-module dependency --> <dependency>
<groupId>com.example</groupId> <artifactId>domain-model</artifactId>
<version>${project.version}</version> </dependency> <!-- Version from parent BOM import -->
```

```
<dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter</artifactId> </dependency> </dependencies> </project>
```

Reactor Build Order

Maven Reactor determines build order from inter-module dependencies — never from <modules> declaration order

The Maven Reactor analyses inter-module dependencies to determine build order — it does NOT simply follow the order of declarations. If module B declares a dependency on module A, the reactor ensures A is built first. Circular dependencies are not allowed and cause a build failure.

Selective Reactor Builds

```
# Build only service-layer and its dependencies mvn -pl service-layer -am clean install # Build
only modules that depend on domain-model mvn -pl domain-model -amd clean install # Resume
failed build from a specific module mvn -rf data-access clean install # Build multiple specific
modules mvn -pl commons,domain-model clean install # Parallel build with 4 threads mvn -T 4
clean install # Parallel build with 1 thread per CPU core mvn -T 1C clean install
```

Aggregator vs Inheritance — Key Difference

Concern	Aggregation (<modules>)	Inheritance (<parent>)
Purpose	Group modules for reactor build	Share config, versions, plugins
Direction	Parent knows about children	Child knows about parent
Required?	Optional	Optional
Combined?	Yes — same POM can do both	Yes — same POM can do both
Effect	Enables mvn on root to build all	Child inherits parent's config

Inter-Module Test Dependencies

```
<!-- Share test utilities between modules --> <!-- In the providing module
(domain-model/pom.xml), configure maven-jar-plugin: --> <plugin>
<groupId>org.apache.maven.plugins</groupId> <artifactId>maven-jar-plugin</artifactId>
<executions> <execution> <goals><goal>test-jar</goal></goals> </execution> </executions>
</plugin> <!-- In the consuming module (service-layer/pom.xml): --> <dependency>
<groupId>com.example</groupId> <artifactId>domain-model</artifactId>
<version>${project.version}</version> <type>test-jar</type> <scope>test</scope> </dependency>
```

CI-Friendly Versions in Multi-Module

```
<!-- parent/pom.xml --> <version>${revision}${changelist}</version> <properties>
<revision>1.5.0</revision> <changelist>-SNAPSHOT</changelist> </properties> <!-- All child
modules reference parent: --> <parent> <groupId>com.example</groupId>
<artifactId>my-app-parent</artifactId> <version>${revision}${changelist}</version> </parent>
<!-- Child inter-module deps also use ${revision}: --> <dependency>
<groupId>com.example</groupId> <artifactId>domain-model</artifactId>
<version>${revision}${changelist}</version> </dependency> # CI pipeline: override version per
```

```
build ./mvnw clean deploy -Drevision=1.5.0 -Dchangelist=
```

Maven Wrapper in Multi-Module

```
# Generate wrapper in root of multi-module project mvn wrapper:wrapper -Dmaven=3.9.6 # This
creates at root level: # mvnw (Unix wrapper script) # mvnw.cmd (Windows wrapper script) #
.mvn/wrapper/maven-wrapper.properties # .mvn/jvm.config — JVM options for all builds -Xmx2g
-XX:+UseG1GC # .mvn/maven.config — Maven options for all builds --batch-mode
--no-transfer-progress -T 1C
```

50+ Interview Questions & Answers

Q1. What is a Maven multi-module project?

A project with a root aggregator POM (packaging=pom) listing child modules in . The Maven Reactor builds all modules in dependency order with a single command from the root.

Q2. What packaging type must the parent/aggregator POM have?

packaging=pom. This is mandatory — without it, Maven would try to build the root as a JAR/WAR, which would fail.

Q3. What is the difference between aggregation and inheritance in Maven?

Aggregation () groups modules for the reactor build — parent knows about children. Inheritance () shares configuration — child knows about parent. The same POM can serve both roles.

Q4. Can aggregation and inheritance be in separate POMs?

Yes. You can have a 'super parent' for inheritance that is not the reactor aggregator, and a separate aggregator POM that doesn't share any configuration. This is an advanced pattern for very large codebases.

Q5. What does the Maven Reactor do?

Determines the correct build order for multi-module projects by analysing inter-module dependencies. Ensures modules are built in dependency order, regardless of their order in .

Q6. How does Maven determine module build order?

By performing a topological sort on inter-module dependencies. Module A is built before module B if B declares a dependency on A. Declaration order in is NOT used.

Q7. Are circular dependencies between modules allowed?

No. Maven's Reactor detects and rejects circular dependencies with a build error. Modules must form a directed acyclic graph (DAG).

Q8. What does -pl flag do in multi-module builds?

Limits the reactor to specific modules: mvn -pl service-layer clean install. Use comma separation for multiple: -pl module-a,module-b. Can reference by directory or artifactId.

Q9. What does -am flag do?

Also Make — builds all modules that -pl targets depend on. mvn -pl service-layer -am ensures domain-model and commons are also built if service-layer depends on them.

Q10. What does -amd flag do?

Also Make Dependents — builds all modules that depend on the -pl targets. Useful for impact analysis: mvn -pl domain-model -amd shows what else needs rebuilding.

Q11. What does -rf flag do?

Resume From — resumes a failed reactor build from a specific module, skipping already-built modules: mvn -rf data-access clean install.

Q12. How do child modules inherit groupId and version?

If not declared in the child POM, groupId and version are inherited from . ArtifactId is never inherited — each module must declare its own.

Q13. What is the relativePath in ?

Points to the parent POM location. Defaults to ../pom.xml (parent directory). Use empty if parent comes from a repository rather than the local file system.

Q14. What does (empty) mean?

Tells Maven not to look for the parent in the local file system — resolve it from the repository only. Used when the parent POM is a separate released artifact, not a local directory.

Q15. How do you share test utilities between modules?

Configure maven-jar-plugin with the test-jar goal in the providing module. In the consuming module, declare the dependency with test-jartest.

Q16. What is CI-friendly versioning in multi-module projects?

Using ${revision} (and optionally ${sha1}, ${changelist}) placeholders in all module versions. Override at build time: mvn deploy -Drevision=1.5.0. Requires flatten-maven-plugin for deployment.

Q17. Why is flatten-maven-plugin needed for CI-friendly versions?

Maven stores the literal ${revision} in installed/deployed POMs. Consumers can't resolve this without the same properties. flatten-maven-plugin resolves placeholders before installation/deployment.

Q18. How do you build only changed modules in CI?

Use mvn -pl $(changed-modules-script) -am. Tools like Takari Maven's smart builder or the maven-incremental-build plugin detect changed modules. Alternatively, use Gradle's build caching features.

Q19. What is the .mvn directory used for in multi-module projects?

.mvn/jvm.config — sets JVM arguments for the Maven process. .mvn/maven.config — sets default Maven command-line options (e.g., --batch-mode, -T 1C). Both apply to all modules.

Q20. How do parallel builds work in multi-module projects?

mvn -T 4 or -T 1C (threads per CPU). Modules without dependencies between them build concurrently. Maven 3.x supports parallel module builds but not parallel goal execution within a module.

Q21. What can break parallel builds?

Modules writing to the same shared resource (e.g., properties file, generated code directory), plugins that are not thread-safe, and missing -am flags causing build-order violations.

Q22. How do you configure properties specific to a module?

Define in the child module's pom.xml. These override parent properties for that module only. Properties cascade from parent to child but not between siblings.

Q23. What is a flat multi-module layout?

All modules at the same directory level as the parent, rather than nested inside a parent directory. Parent references them with ../module-name. Common in older projects.

Q24. How do you exclude a module from the reactor build?

Remove it from in the aggregator POM. For conditional exclusion, use profiles: wrap the declarations in a and control activation via -P flag or properties.

Q25. What is the maven-flatten-plugin (flatten) vs versions:set for multi-module versioning?

flatten-maven-plugin resolves CI-friendly placeholders in published POMs. versions:set updates literal version strings in all module POMs. They serve different purposes — versions:set for release bumps, flatten for CI.

Q26. What happens when you run mvn install on the parent POM?

The reactor installs all sub-modules to the local repository in dependency order. Each sub-module artifact (JAR/WAR) plus its POM is installed to ~/.m2.

Q27. How do you skip a specific module in a multi-module build?

mvn clean install -pl !module-name (negation with !). Or configure true in the module's plugin config (e.g., ${skipModule} with -DskipModule=true).

Q28. What is the difference between the root POM and the parent POM?

They can be the same POM (most common) or separate. The root POM is where you run Maven from. The parent POM is the inheritance target. In very large projects, they may differ.

Q29. How do you reference another module as a dependency?

Use the same GAV coordinates (same groupId, the module's artifactId, same version — typically ${project.version}). Maven resolves this from the local repo after the providing module is built.

Q30. What is the build reactor summary printed after a multi-module build?

BUILD SUCCESS/FAILURE for each module with timing. Shows which modules succeeded, which failed, and which were skipped. Helps diagnose partial failures.

Q31. How does mvn clean work in a multi-module project?

Runs the clean lifecycle on all reactor modules in order, deleting each module's target/ directory.

Q32. What is the spring-boot-maven-plugin's repackage goal doing in a multi-module web module?

Takes the compiled WAR/JAR from the current module, adds the Spring Boot launcher and embeds all dependencies, creating an executable artifact. Other modules remain as thin JARs.

Q33. How do you ensure consistent plugin versions across all modules?

Define all plugin versions in the parent POM's . Child modules activate plugins without specifying versions — they inherit from pluginManagement.

Q34. What is the reactor project list (-pl) syntax for module paths?

Can use relative directory path: -pl service/service-layer or artifactId: -pl service-layer. Use comma separation for multiple. Prefix with ! to exclude: -pl !service-layer.